

Paginate a list

Send a long list one page at a time, with numbers that say where you are.

The big idea

An API with 50,000 tasks can't send them all at once: the response would be huge and slow. So it sends **pages**: a small slice, plus a few numbers that say where you are.

You already do this with `df.head(10)` and `df.iloc[10:20]`. Pagination is the same idea, done by the server, with the **client** choosing the chunk.

What your code receives

The client writes `?page=2&page_size=10`. The tests call your function and pass these in:

```
ITEMS = list(range(1, 43))    # [1, 2, 3, ..., 42] -> 42 items

paginate(ITEMS, page=1, page_size=10)
paginate(ITEMS, page="2", page_size="20")    # text, like a real URL
paginate(ITEMS, page="abc", page_size=10)    # must raise ValueError
paginate(ITEMS, page=6, page_size=10)        # ValueError("Invalid page.")
paginate(list(range(200)), page=1, page_size=1000, max_page_size=50)
paginate([], page=1)            # an empty list
```

What your function must return

`paginate(ITEMS, page=1, page_size=10)` gives one page plus **metadata**:

```
{
    "count": 42,           # how many items exist in total
    "page": 1,            # which page this is (a number, not "1")
    "total_pages": 5,     # 42 items at 10 per page -> 5 pages
    "next_page": 2,       # None on the last page
    "previous_page": None, # None on page 1
    "results": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
}
```

New words

pagination

Splitting a long list into pages and returning one at a time.

page / page size

Which chunk (page 1 is first) / how many items per chunk.

metadata

Facts about the data, like `count` and `total_pages`.

clamp

Push a number back inside a limit: too big, use the limit.

exception

An error that stops the code, like `ValueError`.

raise

Make an exception happen on purpose: "this input is wrong".

Tools: int, try / except, raise

```
def check_age(value):          # a number typed into a form
    try:
        age = int(value)
    except (TypeError, ValueError):
        raise ValueError("age must be a whole number")
    if age < 0:
        raise ValueError("age cannot be negative")
    return age
print(check_age("30"))
check_age("old")
```

OUTPUT

30

ValueError: age must be a whole number

Tools: min, max and math.ceil

`min` clamps a number to a limit; `math.ceil` rounds **up**.

```
import math

print(min(80, 25))      # no more than 25
print(max(1, 0))        # at least 1
print(math.ceil(23 / 5)) # 23 / 5 is 4.6 -> 5 pages
```

OUTPUT

```
25
1
5
```

Rounding down vs rounding up

`count // page_size`

- Rounds **down**
- `42 // 10` gives 4
- Page 5 with `[41, 42]` disappears
- Wrong for counting pages

`math.ceil(count / page_size)`

- Rounds **up**
- `math.ceil(42 / 10)` gives 5
- A half-full last page still counts
- Wrap in `max(1, ...)` for empty lists

Pattern: cut out one page

```
songs = ["s1", "s2", "s3", "s4", "s5", "s6", "s7"]
page, page_size, last_page = 2, 3, 3
start = (page - 1) * page_size                # (2 - 1) * 3 = 3
print(songs[start:start + page_size])         # stops BEFORE 6
print(songs[6:9])                            # past the end: shorter
print(page + 1 if page < last_page else None) # next page
print(page - 1 if page > 1 else None)         # previous page
```

OUTPUT

```
['s4', 's5', 's6']
['s7']
3
1
```

Your task, step by step

- 1 Turn `page` into a number with `int`; raise `ValueError` if bad or below 1 → **test 5**
- 2 Do the same for `page_size` (a small helper saves writing it twice) → **tests 4-5**
- 3 Clamp first: `page_size = min(page_size, max_page_size)` → **test 6**
- 4 `count = len(items); total_pages = max(1, math.ceil(...))` → **tests 1, 8**
- 5 If `page > total_pages`, raise `ValueError("Invalid page.")` → **test 7**
- 6 Slice from `(page - 1) * page_size`; work out next and previous → **tests 1-3**
- 7 Return one dict with all six keys, using the **number** `page` → **tests 1-8**

Common mistakes

- Catching only `ValueError: int(None)` raises a `TypeError`
- Returning the original `page: "2"` must come back as `2`
- Slicing from `page * page_size`: page 1 must start at 0
- Using `//` for pages: 42 items at 10 per page loses 41 and 42
- Counting pages **before** clamping `page_size`

Where you'll use this

A shop's mobile app shows products in a scrolling list. Each time you reach the bottom, it asks `GET /api/products/?page=3` and adds 10 more.

- DRF's `PageNumberPagination` does this: `count, next, previous, results`
- This project sets it in `core/pagination.py`: `page_size = 10, max_page_size = 50`
- A page past the end becomes a `404` with `"Invalid page."`

Check yourself

1. Why does `?page=2` arrive as the string `"2"`?
2. Which two errors can `int(value)` raise, and when?
3. Where does page 3 start with 10 items per page?
4. Why cap `page_size` at a maximum?

Keep learning

[Pagination](#)

Django docs · docs · beginner. Start here: short examples of Django's Paginator showing pages, counts and has_next, the same ideas you implement.

[math.ceil](#)

Python docs · docs · beginner. The function that rounds up, so 21 items at 10 per page gives 3 pages, not 2.

[Pagination: PageNumberPagination](#)

Django REST framework · docs · intermediate. Shows DRF's response shape (count, next, previous, results) and page_size limits that your function copies.